LeetCode Practice

3 Hard: Linked List

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

3
HARD

Patterns (in order)

1. Linked List (3)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

3 questions | tap a title to jump to it

Hard questions (3) Go >

Linked List (3) Go >

H 1. Reverse Nodes in k-Group LC 25

H 2. All O'one Data Structure LC 432

H 3. LFU Cache LC 460

Hard questions (3)

HARD | TOPIC 1 OF 1

Linked List

3 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Reverse Nodes in k-Group LC 25
2. All O'one Data Structure LC 432
3. LFU Cache LC 460

HARD | Linked List | LeetCode #25 | 1 of 3 | 1/3

Reverse Nodes in k-Group

QUESTION

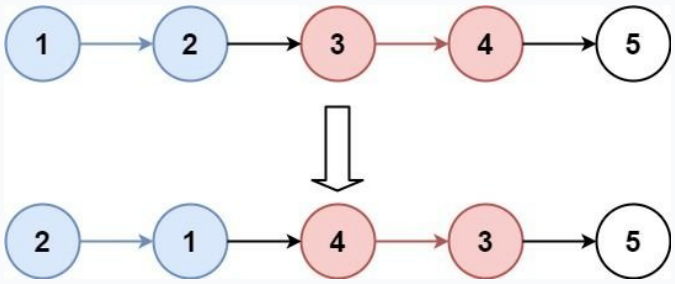
Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed.

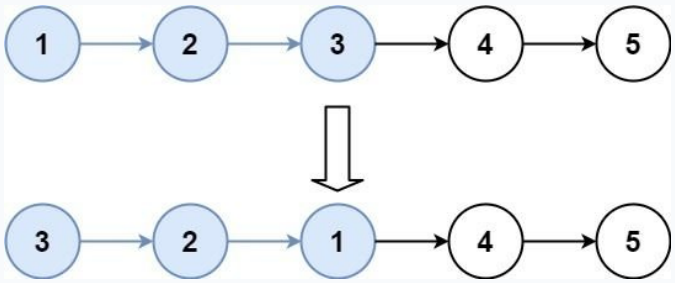
Example 1

Input	`head = [1,2,3,4,5], k = 2`
Output	`[2,1,4,3,5]`



Example 2

Input	`head = [1,2,3,4,5], k = 3`
Output	`[3,2,1,4,5]`



CONSTRAINTS

- The number of nodes in the list is n.
- 1 <= k <= n <= 5000
- 0 <= Node.val <= 1000
- Follow-up: Can you solve the problem in O(1) extra memory space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reverse every k consecutive nodes in the linked list.

If remaining nodes < k, leave them as-is. Return the new head.

O(1) extra space — no copying values. Right?"

"A few quick questions:

PHASE 1 CLARIFY

k can equal 1 (no-op)? Yes. k can equal list length? Yes — reverse everything."

"Let me walk: [1,2,3,4,5], k=2 -> [2,1,4,3,5] ok. k=3 -> [3,2,1,4,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
put the nodes in an array
reverse each block of k in the array (leave a short
    tail)
relink in array order
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear time but O(n) extra space. Counting k nodes ahead and reversing the block in place with pointers is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary value list.

In-place pointer reversal group by group — O(1) extra space:

First check if k nodes remain; if not, stop.

Reverse k nodes using the 3-pointer technique, then link to the next group.

Time: O(n). Space: O(1)."

Key insight: checking that k nodes remain before reversing avoids partial reversal that must be undone — only complete groups are reversed, preserving the tail intact.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _25_ReverseKGroup:
    def reverseKGroup(self, head, k):
        def has_k(node, k):
            for _ in range(k):
                if not node: return False
                node = node.next
            return True

        def reverse_k(head, k):
            prev, cur = None, head
            for _ in range(k):
                nxt = cur.next; cur.next = prev; prev, cur
 = cur, nxt
            return prev

        dummy = ListNode(0, head)
```

```
        group_prev = dummy
        while has_k(group_prev.next, k):
            group_start = group_prev.next
            group_end = group_start
            for _ in range(k - 1): group_end = group_end.n
ext
            next_group = group_end.next
            group_end.next = None
            group_prev.next = reverse_k(group_start, k)
            group_start.next = next_group
            group_prev = group_start
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace reverseKGroup(1->2->3->4->5, k=2)."

Check if k nodes remain from head. Group [1,2]: reverse -> 2->1. Then recurse on [3,4,5].

Group [3,4]: reverse -> 4->3. Recurse on [5]. 5 < k=2 -> return 5 unchanged.

Stitch: 2->1->4->3->5. Result ok

Reverse k nodes: curr=head. For k steps: nxt=curr.next; curr.next=prev; prev=curr; curr=nxt.

Returns (new_head=prev, new_tail=old_head) for stitching.

k=3 on [1..5]: [1,2,3]->3->2->1. [4,5]<3 -> unchanged. Result: 3->2->1->4->5 ok

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each node processed in its reversal group exactly once	**O(1)**

k=2 is exactly Swap Nodes in Pairs (#24).

Follow-up: k=n reverses the entire list — identical to Reverse Linked List (#206).

Follow-up: reverse every other group of k — add a skip_k step between reverse steps.

HARD | Linked List | LeetCode #432 | 2 of 3 | 2/3

All O'one Data Structure

QUESTION

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the AllOne class:

CONSTRAINTS

- AllOne() Initializes the object of the data structure.
- inc(String key) Increments the count of the string key by 1. If key does not exist in the data structure, insert it with count 1.
- dec(String key) Decrements the count of the string key by 1. If the count of key is 0 after the decrement, remove it from the data structure. It is guaranteed that key exists in the data structure before the decrement.
- getMaxKey() Returns one of the keys with the maximal count. If no element exists, return an empty string "".
- getMinKey() Returns one of the keys with the minimum count. If no element exists, return an empty string "".

Note that each function must run in O(1) average time complexity.

- 1 <= key.length <= 10
- key consists of lowercase English letters.
- It is guaranteed that for each call to dec, key is existing in the data structure.
- At most 5 * 10^4 calls will be made to inc, dec, getMaxKey, and getMinKey.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a data structure with inc(key), dec(key), getMaxKey(), getMinKey() — all O(1).

inc adds key with count 1 if absent; dec removes if count drops to 0. Right?"

"A few quick questions:

getMaxKey/getMinKey return any key with max/min count? Yes — any valid key.

What if the data structure is empty? Return an empty string."

"Let me walk: inc('a'),inc('b'),inc('b'),inc('c'),inc('c'),inc('c').

`getMaxKey()->'c'(3). getMinKey()->'a'(1) ok`

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep a map from key to count
inc / dec: update the count
getMaxKey / getMinKey: scan the whole map
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) for getMax / getMin	O(n)

WHY IT IS SLOW

The max and min are found by scanning every key. A doubly linked list of buckets ordered by count (each holding its keys), plus a map from key to bucket, makes all four operations O(1).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) scan for getMax/getMin.

Doubly linked list of buckets (each bucket holds a count and a set of keys at that count)

+ a hashmap {key -> bucket_node}.

inc: move key from bucket c to c+1. dec: move key to c-1.

getMax: any key from the tail bucket. getMin: any key from the head bucket.

All O(1) amortized. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _432_AllOoneDataStructure:
    class _Bucket:
        def __init__(self, count):
            self.count = count; self.keys = set()
            self.prev = self.next = None

    def __init__(self):
        self._key_bucket = {}
        self._head = self._Bucket(0); self._tail = self._B
ucket(float('inf'))
        self._head.next = self._tail; self._tail.prev = se
lf._head

    def _insert_after(self, node, count):
        b = self._Bucket(count)
        b.prev = node; b.next = node.next
        node.next.prev = b; node.next = b
        return b

    def _remove_bucket(self, b):
        b.prev.next = b.next; b.next.prev = b.prev

    def inc(self, key):
        if key not in self._key_bucket:
            if self._head.next.count != 1:
                self._insert_after(self._head, 1)
            self._head.next.keys.add(key)
            self._key_bucket[key] = self._head.next
        else:
            cur = self._key_bucket[key]
            nxt_count = cur.count + 1
            if cur.next.count != nxt_count:
                self._insert_after(cur, nxt_count)
            cur.next.keys.add(key)
            self._key_bucket[key] = cur.next
            cur.keys.discard(key)
            if not cur.keys: self._remove_bucket(cur)
```

```
    def dec(self, key):
        cur = self._key_bucket[key]
        prev_count = cur.count - 1
        if prev_count == 0:
            del self._key_bucket[key]
        else:
            if cur.prev.count != prev_count:
                self._insert_after(cur.prev, prev_count)
            cur.prev.keys.add(key)
```

```
            self._key_bucket[key] = cur.prev
        cur.keys.discard(key)
        if not cur.keys: self._remove_bucket(cur)

    def getMaxKey(self):
        return next(iter(self._tail.prev.keys), '') if sel
f._tail.prev != self._head else ''

    def getMinKey(self):
        return next(iter(self._head.next.keys), '') if sel
f._head.next != self._tail else ''
```

PHASE 5 TEST & DEBUG

"Let me trace AllOne: inc('a'),inc('a'),inc('b'),getMaxKey(),getMinKey()."

Doubly linked list of frequency nodes + hash maps.

inc('a'): key_count={a:1}. freq_node[1]=node(keys={a}). List: head1tail.

inc('a'): key_count={a:2}. Move a from freq 1 to freq 2. List: head12tail. freq 1 empty -> remove. List: head2tail.

inc('b'): key_count={b:1}. freq_node[1]=node(keys={b}). List: head12tail.

getMaxKey(): tail.prev=freq 2->any key='a' ok

getMinKey(): head.next=freq 1->any key='b' ok

WHY linked list of freq nodes: O(1) min/max via head/tail. O(1) increment via splice.

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(1): set operations + DLL pointer updates. Space O(n).

Follow-up: if we also need getMedianKey, we'd need a balanced BST instead of a DLL.

Follow-up: LFU Cache (#460) uses the same bucket-DLL structure.

HARD | Linked List | LeetCode #460 | 3 of 3 | 3/3

LFU Cache

QUESTION

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

CONSTRAINTS

- LFUCache(int capacity) Initializes the object with the capacity of the data structure.
- int get(int key) Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- void put(int key, int value) Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (i.e., two or more keys with the same frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in O(1) average time complexity.

- $1 <= capacity <= 10^4$
- $0 <= key <= 10^5$
- $0 <= value <= 10^9$
- At most $2 * 10^5$ calls will be made to get and put.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

LFU Cache: get(key)->val or -1, put(key,val). Evict the least frequently used.

On ties, evict the least recently used among the LFU items. All O(1). Right?"

"A few quick questions:

Does get increment frequency? Yes. Put on existing key — update value and increment freq?"

"Let me walk: LFUCache(2): put(1,1),put(2,2),get(1)->1 (freq[1]=2).

```
put(3,3) evicts key 2 (freq=1, LRU among min-freq).
  get(2)->-1 ok
```

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep a list of (key, value, use count, last used
    time)
get / put: scan the list for the key
on overflow:
    scan for the least frequently (then least
        recently) used item and remove it
```

TIME	SPACE
O(n) per operation	O(n)

WHY IT IS SLOW

Every operation scans the list. A map from key to node plus a map from frequency to a doubly linked list of keys (with the minimum frequency tracked) makes get and put O(1).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) scan for the LFU key.

Three structures for O(1):

1. key_map: {key -> (val, freq)}

2. freq_map: {freq -> OrderedDict(key->None)} — LRU order within same freq

3. min_freq: current minimum frequency

get: update freq, move key to freq+1 bucket. put: evict from freq_map[min_freq].

All ops O(1). Space O(capacity)."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict, OrderedDict
class _460_LFUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.key_map = {}
        self.freq_map = defaultdict(OrderedDict)
        self.min_freq = 0

    def _update(self, key):
        val, freq = self.key_map[key]
        del self.freq_map[freq][key]
        if not self.freq_map[freq] and freq == self.min_fr
eq:
            self.min_freq += 1
        self.key_map[key] = (val, freq + 1)
        self.freq_map[freq + 1][key] = None

    def get(self, key):
```

```
        if key not in self.key_map: return -1
        self._update(key)
        return self.key_map[key][0]

    def put(self, key, value):
        if self.cap == 0: return
        if key in self.key_map:
            self._update(key)
            val, freq = self.key_map[key]
            self.key_map[key] = (value, freq)
        else:
            if len(self.key_map) == self.cap:
                evict_key, _ = self.freq_map[self.min_freq
].popitem(last=False)
                del self.key_map[evict_key]
            self.key_map[key] = (value, 1)
            self.freq_map[1][key] = None
            self.min_freq = 1
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

LFUCache(2): put(1,1): key_map={1:(1,1)},freq_map={1:{1}},min=1.

put(2,2): key_map={1:(1,1),2:(2,1)},freq_map={1:{1,2}},min=1.

get(1): _update(1)->freq_map={1:{2},2:{1}},min=1. return 1 ok.

put(3,3): evict freq_map[1]->evict key2. key_map={1:(1,2),3:(3,1)},min=1. get(2)->-1 ok.

OrderedDict preserves insertion order so popitem(last=False) evicts the LRU

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

Doubly linked list per freq bucket: head.next=MRU (Most Recently Used).next=...LRU. Remove: node.prev.next=node.next; node.next.prev=node.prev. After get(key): curr=node, move to freq[curr_freq+1] list head.

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(1). Space O(capacity).

min_freq only ever decreases on put (new key starts at freq=1) — always correct.

Follow-up: LRU Cache (#146) — simpler, only recency matters.

Follow-up: add TTL — track expiry timestamps and remove in background or on access.